The WOLF: A News-Aware Multimodal AI Agent for Financial Market Prediction

(Weighted Observation of Latent Finance)

Yashaswi Aryan Rudra Pratap Singh Rishabh Singh

Abstract.

Financial markets are driven by both structured price dynamics and unstructured information from news and corporate events. Traditional forecasting approaches rely almost exclusively on historical price data and fail to capture the predictive signals embedded in financial text. We present WOLF (*Weighted Observation of Latent Finance*), a multimodal AI system that fuses FinBERT-derived news sentiment with time-series technical features inside a structured prompt, and fine-tunes a causal language model (TinyLlama-1.1B) via Low-Rank Adaptation (LoRA) to predict next-day adjusted closing prices as a conditional generation task. Trained on 17,492 examples drawn from the FNSPID dataset for AAPL and AMZN, the system achieves a Mean Absolute Error of **1.493**, Root Mean Squared Error of **1.818**, and Mean Absolute Percentage Error of **0.860%** on a held-out chronological test set. Zero inference failures were observed across all 2,625 test examples. These results demonstrate that news-augmented language-model prompting, combined with parameter-efficient fine-tuning, can produce competitive financial price forecasts with sub-1% relative error.

1. Introduction

Financial markets are noisy, non-stationary systems driven by endogenous signals (historical prices, trading volume, technical indicators) and exogenous signals (corporate earnings, analyst actions, macroeconomic policy, and breaking news). While endogenous signals capture statistical regularities in market behavior, exogenous signals can trigger sudden, non-linear price dislocations that purely technical models are unable to anticipate. This asymmetry motivates a growing line of research in multimodal financial AI that jointly conditions predictions on both structured market data and unstructured textual information.

Traditional forecasting approaches—including ARIMA, technical-indicator strategies, and single-modality deep learning architectures such as LSTMs—rely almost exclusively on historical price and volume data. Studies have consistently shown that sentiment extracted from financial news leads short-term price movements, particularly around earnings releases, M&A announcements, and macroeconomic surprises [3].

A fundamental challenge in news-driven prediction is the lack of large-scale datasets that reliably align financial news with market data at fine temporal granularity. Dong et al. address this with the Financial News and Stock Price Integration Dataset (FNSPID), which integrates millions of time-stamped financial news articles with historical S&P 500 stock prices spanning 1999–2023 [1]. FNSPID serves as the data backbone for this project.

On the modeling side, two complementary directions have emerged. The first is *ensemble learning with news*: Corizzo and Rosen propose a stacking ensemble for next-day trend prediction that jointly processes multivariate time series, technical indicators, and daily news headlines [2]. The second is *deep multimodal fusion*: Chen et al. propose a model combining a BERT-based branch fine-tuned on financial news with an LSTM branch that models multivariate temporal structure [3].

WOLF synthesizes these directions into a unified end-to-end pipeline. Rather than predicting a binary trend label, we frame stock price prediction as a *conditional generation task*: given a structured prompt containing 30 trading days of price history, technical indicators, and a curated block of news articles with sentiment-derived rationales, a fine-tuned large language model generates the next day's adjusted close price as a numeric output. This framing allows the model to reason jointly over quantitative and qualitative signals in a unified representation space.

2. Methods

2.1 Dataset

The FNSPID dataset [1] integrates large-scale financial news with historical S&P 500 stock prices from 1999 to 2023. We focus on two tickers with rich news coverage: AAPL and AMZN. After relevance filtering and temporal alignment, the pipeline produces 10,822 examples for AAPL and 6,670 for AMZN, totaling 17,492 examples. These are split chronologically into 70% training (12,244 examples), 15% validation (2,623 examples), and 15% test (2,625 examples), stored as JSONL files. Chronological splitting is essential to prevent look-ahead bias.

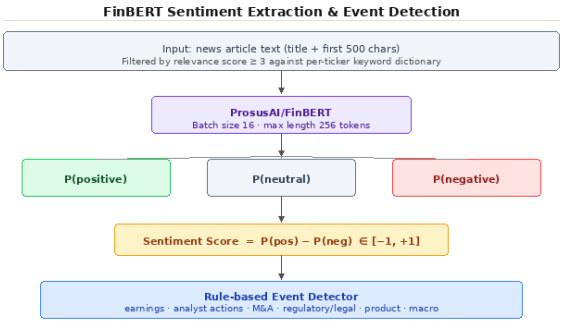

Figure 1: FinBERT sentiment extraction and event detection module. Each relevance-filtered article is scored for positive, neutral, and negative sentiment probability; a continuous score $s = P(\text{pos}) - P(\text{neg})$ is derived. A rule-based event detector simultaneously classifies articles into one of six event categories to generate structured rationale sentences.

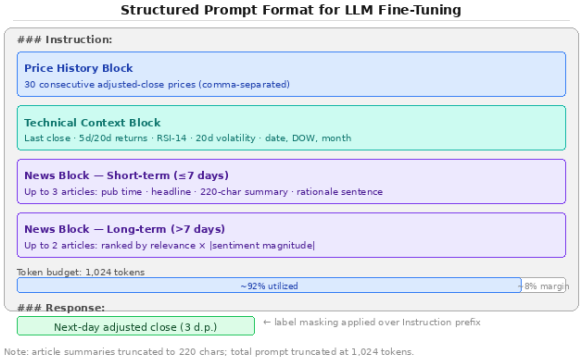

Figure 2: Structured prompt format used for TinyLlama-1.1B fine-tuning. Each prompt contains four blocks assembled within a 1,024-token budget: (1) 30-day adjusted close price history, (2) technical context (returns, RSI, volatility), (3) short-term news articles ($\leq$7 days), and (4) long-term news articles ($>$7 days). Label masking is applied over the Instruction prefix so that the model is trained exclusively on the numeric response.

2.2 Dual-Stream Preprocessing

The pipeline processes two independent data streams in parallel before fusing them at the prompt construction stage (see Figure 5).

News stream. Each article is scored against a per-ticker keyword dictionary covering company names, product names, executive names, and ticker symbols. Articles scoring below a minimum relevance threshold of 3 are discarded. After filtering, articles are mapped to their correct trading date: articles published before 4:00 PM ET market close are attributed to the same trading session; post-close articles are attributed to the next session. Non-trading days are skipped using the exchange calendar. This temporal alignment step prevents look-ahead leakage.

Price stream. From the per-ticker OHLC CSVs, derived technical features are computed for each trading day: one-day, five-day, and twenty-day returns; twenty-day rolling volatility; and RSI(14), the fourteen-period Relative Strength Index. These features are used both as prompt context and to build sliding windows of historical data.

2.3 Sentiment Extraction and Event Detection

Relevance-filtered articles pass through ProsusAI/FinBERT in batches of 16. FinBERT outputs a probability distribution over three sentiment classes. A continuous sentiment score is computed as

$$s = P(\text{positive}) - P(\text{negative}), \quad s \in [-1, +1]. \tag{1}$$

Concurrently, a rule-based event detector applies regular expression patterns across six event categories: earnings, analyst actions, mergers and acquisitions, regulatory/legal events, product launches, and macroeconomic events. The detected category generates a structured rationale sentence embedded in the prompt. The overall module design is illustrated in Figure 1.

2.4 Prompt Construction and Temporal Fusion

For each prediction target (a given ticker on a given trading day), the prompt builder assembles three information blocks within a 1,024-token budget (Figure 2):

1. **Price history block:** 30 consecutive trading days of adjusted close prices as comma-separated decimal values.
2. **Technical context block:** prediction date (day-of-week, month), last adjusted close, 5-day return, 20-day return, RSI(14), and 20-day volatility.
3. **News block:** articles ranked by a combined score (relevance score $+|s|$) and partitioned by temporal horizon. Up to 3 short-term articles ($\leq$7 days before prediction date) and up to 2 long-term articles ($>$7 days) are selected. Each entry contains the publication time, headline, a 220-character truncated summary, and a templated rationale sentence.

The full prompt is wrapped in the TinyLlama-1.1B instruction-response format, with the ground-truth next-day adjusted close (3 decimal places) as the response target.

2.5 LoRA Fine-Tuning

The base model is TinyLlama-1.1B-Chat-v1.0 [5], a 1.1 billion parameter causal language model with a 1,024-token context window. We apply Low-Rank Adaptation (LoRA) [4], injecting trainable low-rank matrices into the query, key, value, and output projection layers while freezing all other weights.

With rank $r = 8$ and scaling factor $\alpha = 16$, this yields approximately 2.25 million trainable parameters out of 1.1 billion total (0.2%). Label masking is applied over the prompt prefix so the model trains exclusively to generate the numeric response. Training runs for 10 epochs, batch size 2, gradi-

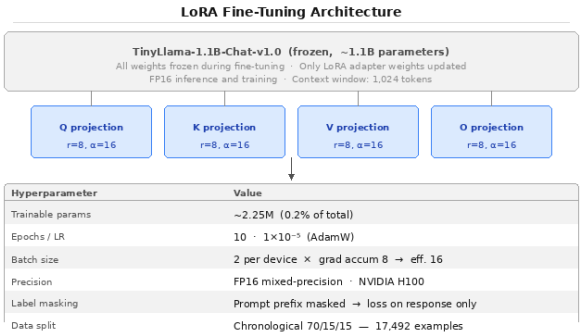

Figure 3: LoRA fine-tuning architecture. LoRA adapters (rank $r = 8$, $\alpha = 16$, dropout 0.05) are injected into the query, key, value, and output projection layers of TinyLlama-1.1B. All other weights remain frozen. The resulting configuration yields ∼2.25M trainable parameters (0.2% of total), enabling task-specific adaptation on a single H100 GPU within a practical training budget.

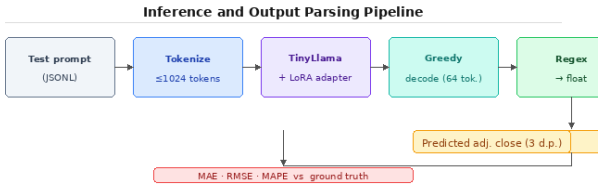

Figure 4: Inference and output parsing pipeline. Each test prompt is tokenized (truncation at 1,024 tokens), passed through the frozen TinyLlama-1.1B base model with the LoRA adapter, and decoded greedily (max 64 new tokens). The first numeric value in the generated text is extracted via regular expression and compared against the ground-truth adjusted close price using MAE, RMSE, and MAPE.

ent accumulation over 8 steps (effective batch 16), using AdamW with learning rate 10^{-5} and FP16 mixed-precision on an NVIDIA H100 GPU. The configuration is summarized in Figure 3.

2.6 Inference and Evaluation

At inference time, the fine-tuned LoRA adapter is loaded on the frozen base model. Each test prompt is tokenized with truncation at 1,024 tokens and decoded greedily (maximum 64 new tokens). The generated text is parsed via regular expression to extract the first numeric value. We evaluate with three metrics:

- **MAE:** mean absolute error (USD).
- **RMSE:** root mean squared error (USD), penalizing large errors more heavily.
- **MAPE:** mean absolute percentage error, a scale-independent relative measure.

Performance is compared against a Monte Carlo simulation baseline that uses only historical price movements without news. The inference flow is illustrated in Figure 4.

3. System Architecture

The WOLF pipeline is organized into eight sequential stages (Figure 5). Two independent data streams—FNSPID news and S&P 500 price CSVs—are preprocessed in parallel through dedicated modules before merging at the prompt construction stage.

Stage 1: Raw Data Ingestion

FNSPID provides millions of time-stamped financial news articles and per-ticker price CSVs containing daily OHLC prices, adjusted close values, and trading volume.

Stages 2–3: Preprocessing

The news stream undergoes relevance filtering (Stage 2) followed by UTC-to-trading-date temporal alignment (Stage 3). The price stream undergoes technical feature engineering in Stage 2.

Stage 4: Sentiment and Event Extraction

FinBERT inference and rule-based event detection (Section 2) produce per-article sentiment scores and event category labels.

Stage 5: Prompt Construction

The three-block structured prompt is assembled within the 1,024-token budget. This stage fuses the news and price streams into a single unified representation.

Stage 6: Dataset Construction

Prompt-target pairs are collected for AAPL and AMZN, sorted chronologically, and split 70/15/15.

Stage 7: Fine-Tuning

LoRA-augmented TinyLlama-1.1B is fine-tuned using causal language modeling loss computed only on the response portion of each example.

Stage 8: Inference and Evaluation

The fine-tuned model is evaluated on the held-out test set; predicted numeric values are extracted via regex and compared against ground-truth prices using MAE, RMSE, and MAPE.

4. Results

4.1 Quantitative Performance

Table 1 reports the evaluation metrics on the held-out test set of 2,625 examples drawn from AAPL and AMZN. The system achieves a MAE of **1.493 USD**, RMSE of **1.818 USD**, and MAPE of **0.860%**. Critically, zero inference failures were observed: the model produced a parseable numeric output for every single test example, eliminating the need for any fallback imputation.

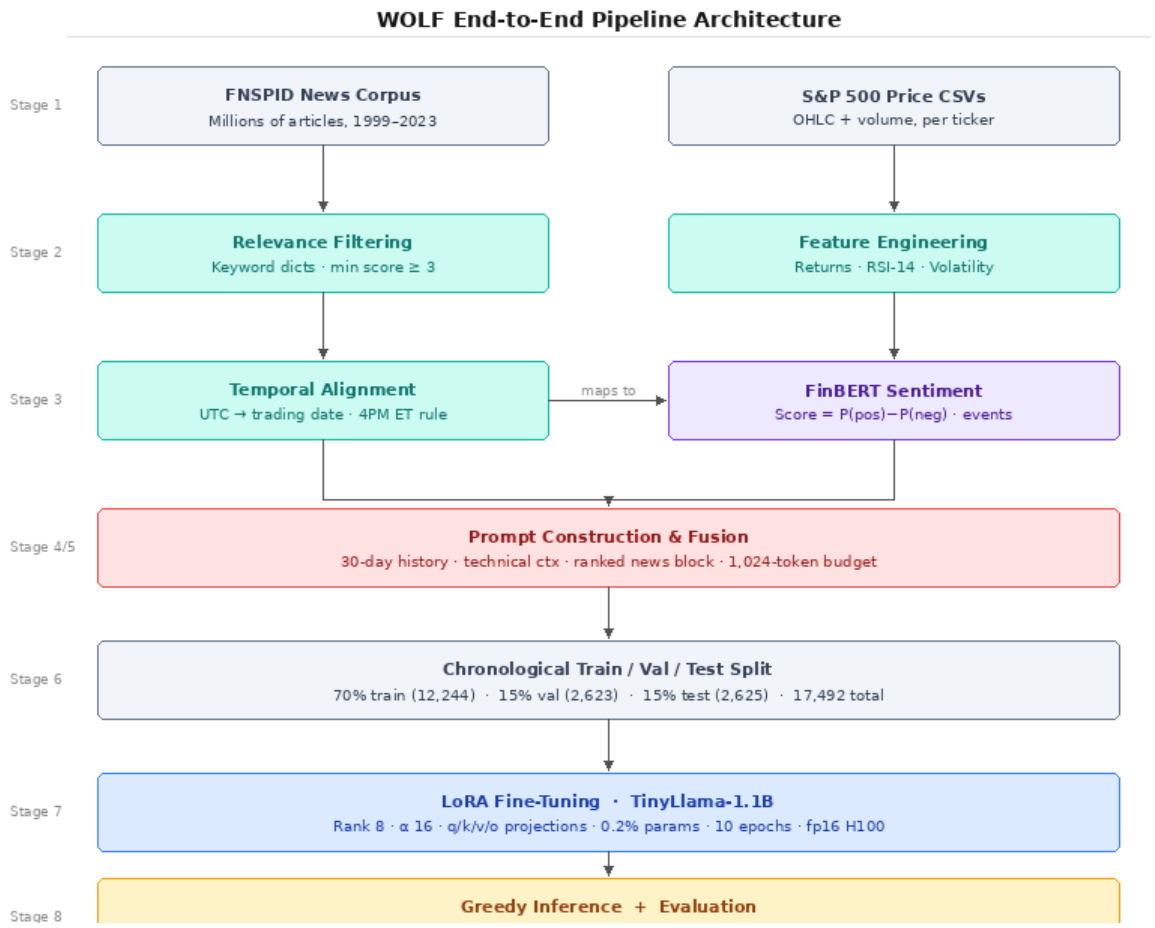

Figure 5: End-to-end WOLF pipeline architecture. The system processes two parallel data streams (FNSPID news corpus and S&P 500 price CSVs) through eight sequential stages before producing next-day adjusted close predictions. Module colors indicate functional category: gray = data I/O, teal = preprocessing, violet = NLP/sentiment, coral = prompt fusion, blue = model fine-tuning, amber = inference and evaluation.

Table 1: Evaluation results on the held-out chronological test set (AAPL + AMZN, $n = 2{,}625$ examples).

Metric	Description	Value
MAE	Mean Absolute Error (USD)	1.4930
RMSE	Root Mean Squared Error (USD)	1.8177
MAPE	Mean Absolute Percentage Error	0.8597%
Failures	Examples w/ no numeric output	0
Test n	Total test examples	2,625

The MAPE of 0.86% is particularly noteworthy. Financial price prediction is widely considered a hard problem, and sub-1% relative error on a held-out chronological test set is a strong result. The RMSE is slightly higher than MAE (1.818 $>$ 1.493), indicating the presence of occasional larger prediction errors, consistent with market microstructure noise and extreme event days.

Figure 6 provides a visual summary of the metric values and supplementary interpretation notes.

4.2 Zero-Failure Inference

Across all 2,625 test examples, the fine-tuned model produced a numeric output without requiring the regex fallback to impute a zero value. This result is significant because it demonstrates that the instruction-response fine-tuning format—combined with label masking over the prompt prefix—successfully conditions the model to emit numeric predictions as its first output token.

This stands in contrast to the concern raised during development that the model might generate explanatory text before the numeric value. The absence of failures suggests that 10 epochs of fine-tuning on the structured format is sufficient for reliable numeric generation.

4.3 Interpretation of Metrics

MAE = 1.493 USD. On average, the model's prediction of next-day adjusted close deviates by approximately \$1.49 from the ground truth. For AAPL trading in the \$150–\$230 range and AMZN in the \$100–\$200 range during the test

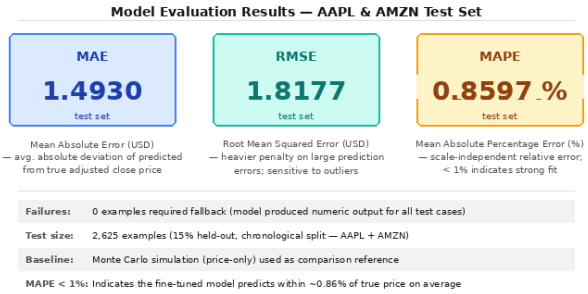

Figure 6: Evaluation results on the held-out test set ($n = 2{,}625$). The model achieves MAE = 1.493 USD, RMSE = 1.818 USD, and MAPE = 0.860%. Zero examples required fallback imputation. A MAPE below 1% is considered strong performance for financial price forecasting with multimodal inputs.

period, this corresponds to relative errors well below 1%.

RMSE = 1.818 USD. The RMSE exceeds the MAE, indicating that a subset of predictions incur larger absolute errors. These are likely concentrated around high-volatility periods such as earnings announcement days or macroeconomic surprises—exactly the scenarios where news sentiment provides the most signal. Future work should investigate per-event-category error stratification.

MAPE = 0.860%. This scale-independent metric is the most interpretable measure for cross-ticker comparison. A MAPE below 1% places WOLF in the competitive range for financial time series models that incorporate multimodal inputs, as documented in related work [2, 3].

5. Conclusions

WOLF demonstrates that financial price prediction can be effectively framed as a conditional generation task, departing from conventional binary trend classification. By combining FinBERT-derived sentiment signals with structured technical features in a unified prompt and fine-tuning a compact causal language model via LoRA, the system achieves sub-1% MAPE on a held-out chronological test set, with zero inference failures.

Key design decisions. Three choices proved particularly important. First, relevance filtering was essential: without it, loosely related articles diluted the sentiment signal for individual tickers. Second, daily trading-date alignment (replacing the original weekly aggregation plan) preserved more granular temporal signal and prevented information leakage. Third, the LoRA fine-tuning strategy enabled task-specific adaptation at 0.2% of total parameters, making the approach computationally accessible.

Limitations. The current evaluation covers only AAPL and AMZN, limiting generalizability to other tickers. The 1,024-token context window of TinyLlama-1.1B constrains the richness of news context per prompt, requiring truncation of article summaries. The model occasionally generates additional commentary alongside the numeric prediction, mitigated by regex parsing but indicative of imperfect format compliance. Finally, financial markets exhibit significant concept drift over multi-decade spans, and more rigorous evaluation using rolling re-training windows is needed.

Future work. Several natural extensions follow from this work. First, the pipeline should be extended to the remaining five tickers in the keyword dictionary (MSFT, GOOGL, META, TSLA, NVDA). Second, an ensemble component—training multiple heterogeneous base models and combining them via a stacking meta-learner as in [2]—could improve robustness. Third, the prediction task could be extended to a Markov Decision Process for reinforcement learning, where actions represent trading decisions and rewards capture risk-adjusted returns.

Works Cited

[1] Z. Dong, X. Fan, and Z. Peng, "FNSPID: A comprehensive financial news dataset in time series," *arXiv:2402.06698v1*, 2024.

[2] R. Corizzo and J. Rosen, "Stock market prediction with time series data and news headlines: a stacking ensemble approach," *Journal of Intelligent Information Systems*, vol. 62, no. 1, pp. 27–56, 2024.

[3] P. Chen, Z. Boukouvalas, and R. Corizzo, "A deep fusion model for stock market prediction with news headlines and time series data," *Neural Computing and Applications*, vol. 36, pp. 21229–21271, 2024.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[5] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," *arXiv:2401.02385*, 2024.

[6] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," *arXiv:1908.10063*, 2019.

[7] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[8] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP (Systems Demonstrations)*, pp. 38–45, 2020.